

1. Your Terraform apply fails because the state file is locked. What could cause this, and how do you safely resolve it in a team environment?

What caused it & how to safely resolve it

What really causes this

In real teams, state locks usually happen because:

- Someone ran terraform apply from **their laptop**, lost VPN, closed terminal, or system crashed
- A **CI/CD pipeline got cancelled** midway
- Two pipelines triggered at the same time on the same state
- Azure Storage backend still thinks a process is holding the lease

Terraform isn't broken it's **protecting you from two people destroying infra at once**.

How I safely resolve it (never rush this):

Step 1 – Identify who locked it

```
terraform force-unlock <LOCK_ID>
```

Before running this, I check:

- Azure DevOps / GitHub Actions → any running pipeline?
- Ask in team channel: *“Anyone running Terraform on env-prod?”*

Step 2 – If no one is running Terraform

Then and only then:

```
terraform force-unlock <LOCK_ID>
```

Step 3 – Prevent future locks

In team setups:

- **Only CI/CD can apply**, no local applies
- Use pipeline-level locks (environment approvals)
- Separate state per environment (never shared)

A Terraform state lock usually means another run was interrupted or is still active, so the safe resolution is to confirm no deployment is running and then release the lock without risking concurrent infrastructure changes.

2. Someone manually modified an Azure resource that Terraform manages. How do you detect this drift and bring things back in sync without breaking production?

How to detect & fix without breaking prod

How drift happens in real life:

- Azure portal “quick fix”
- Auto-scaling / policies modifying tags
- Security team adds rules manually
- Someone “just tested something”

Terraform doesn't yell it **quietly remembers**.

Detect drift safely:

terraform plan

This is your **X-ray** no changes applied yet.

For large states:

terraform plan -refresh-only

This **ONLY** checks reality vs state.

Fix without breaking production:

Option A – Accept the manual change

If the change is valid:

terraform apply -refresh-only

Then update code to match reality.

Option B – Revert to Terraform truth

If change is not approved:

terraform apply

Terraform will bring Azure back to declared state.

Option C – Ignore intentional drift

For things Azure changes itself:

```
lifecycle {  
  ignore_changes = [tags, scaling]  
}
```

Plan = detective, Apply = judge

Never skip the detective phase.

This situation is handled by first detecting the drift through a Terraform plan and then deciding whether to update the code or let Terraform restore the original configuration without impacting production.

3. You're using a remote backend with Azure Storage Account + blob container for state. State corruption is suspected. What steps do you take to investigate and recover?

What I actually do in production

First rule: STOP APPLYING

State corruption + apply = disaster.

Step-by-step recovery (real ops flow):

Step 1 – Check backend health

- Storage Account → Blob → container
- Look for:
 - Missing .tfstate
 - Size suddenly 0 KB
 - Recent failed writes

Step 2 – Use versioning (lifesaver)

If blob versioning is enabled:

- Restore last known good version
- Rename corrupted state (never delete)

Step 3 – Validate locally

```
terraform init  
terraform state list  
terraform plan
```

If plan shows **mass destruction**, stop.

Step 4 – Rebuild state if needed

Last resort:

terraform import

Painful, but safer than recreating prod.

Prevention going forward:

- Enable **Blob versioning + soft delete**
- One backend per environment
- CI-only applies

State is the brain of Terraform

Protect it like prod data.

When state corruption is suspected, deployments are paused and the remote state is inspected or restored from storage versioning so Terraform regains an accurate view of existing Azure resources

4.A Terraform plan shows that a critical Azure resource will be recreated, even though no code change was made. How do you debug this?

No code change, still recreation why?

This is one of the **most dangerous Terraform moments**.

Real reasons this happens:

- Azure API changed a computed value
- Provider version upgrade changed behavior
- Immutable fields changed silently (like location, identity)
- Someone imported resource incorrectly
- Resource ID mismatch in state

How I debug safely:

Step 1 – Detailed diff

terraform plan -out=tfplan

terraform show tfplan

Look for:

- forces replacement
- Attribute marked as Computed
- Identity / SKU / subnet changes

Step 2 – Check provider version

```
required_providers {  
  azurerm = {  
    version = "~> 3.XX"  
  }  
}
```

Provider drift is a silent killer.

Step 3 – Inspect state

terraform state show <resource>

Step 4 – If change is harmless

Use:

```
lifecycle {  
  ignore_changes = [attribute]  
}
```

Never apply blindly for critical resources.

Terraform only destroys when it's confused

Your job is to remove confusion, not apply faster.

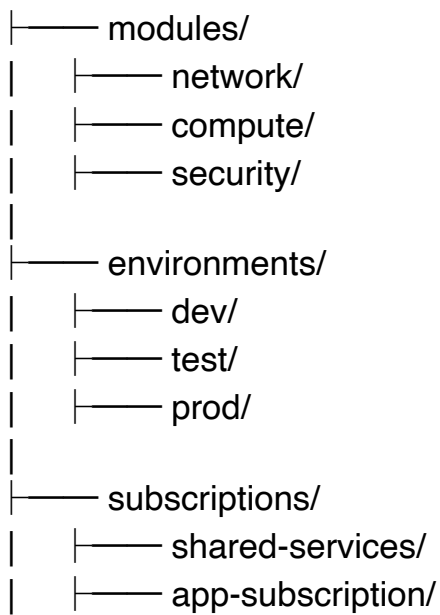
This usually indicates a mismatch between Terraform state, provider behaviour, or Azure-reported values, so the plan must be analyzed carefully before allowing any destructive action.

5.How do you structure Terraform code for a large organization with multiple teams, environments, and Azure subscriptions?

What actually works at scale:

Folder structure (clean + proven)

terraform/



Key principles we follow:

- **One state per environment per subscription**
- **Modules are reusable, environments are thin**
- No hardcoded values everything via variables
- Backend configured per environment

CI/CD flow:

- PR → terraform plan
- Approval → terraform apply
- No local apply for prod (ever)

Azure-specific best practices:

- Separate subscription for networking
- Use management groups
- Central state storage with RBAC
- Use Azure AD auth (no access keys)

Modules are factories, environments are assembly lines

Terraform is structured by separating reusable modules from environment-specific configurations and isolating state per team, environment, and subscription to ensure safe and scalable deployments.

6. You need to deploy the same infrastructure across dev, staging, and prod with minimal duplication. How do you design modules, variables, and environment separation?

How to design modules, variables & environment separation

The real goal:

- Same code
- Different behaviour
- Zero copy-paste
- Safe promotions

proven design:

Thin environments, thick modules

```
modules/  
  vnet/  
  aks/  
  keyvault/
```

```
environments/  
  dev/  
  stage/  
  prod/
```

Each environment folder only has:

- main.tf → calls modules
- variables.tf
- backend.tf
- terraform.tfvars

Environment decides *values*, not *logic*

```
module "vnet" {  
  source    = "../../modules/vnet"
```

```
cidr_block = var.vnet_cidr
env        = var.environment
}
```

Different CIDRs, SKUs, scaling **same module**.

Separate state per environment (non-negotiable)

- One blob container per env
- One backend config per env
- No shared state ever

Modules define WHAT, environments define HOW MUCH

This design keeps Terraform modules reusable and environment folders lightweight, so the same infrastructure is deployed everywhere while only environment-specific values change, not the logic.

7. Terraform apply partially failed, and now Azure resources are in an inconsistent state. What's your recovery approach?

Azure resources are now inconsistent recovery approach

First rule: Don't re-apply immediately

That's how outages happen.

Step-by-step real recovery:

Freeze changes

- Stop pipelines
- No manual fixes in portal yet

Check Terraform's view

terraform plan

This tells you:

- What Terraform thinks exists
- What Azure actually has

Identify the failure type

Scenario	Fix
----------	-----

Resource created, not in state	<code>terraform import</code>
Resource in state, missing in Azure	Re-apply
Half-configured resource	<code>terraform taint</code>
API timeout or throttling	Retry apply

Use -target only for recovery

`terraform apply -target=azurerm_resource.example`
 Never use this as a habit.

When apply fails, Terraform didn't lose control you did
 Slow down, inspect, then act.

When Terraform partially fails, the priority is to stop further changes, understand what Azure created versus what Terraform recorded, and then carefully reconcile state before continuing.

8. How do you handle secrets in Terraform securely when provisioning Azure resources, especially in CI/CD pipelines?

What NOT to do (common mistake):

- Secrets in `.tfvars`
- Secrets in Git
- Outputting secrets

Azure Key Vault is the source of truth

Terraform:

- **Creates** Key Vault
- **Reads** secrets
- **Never stores plaintext**

```
data "azurerm_key_vault_secret" "db_password" {
  name      = "db-password"
  key_vault_id = azurerm_key_vault.kv.id
}
```

CI/CD uses managed identity

- No service principal secrets
- Azure DevOps / GitHub → OIDC
- RBAC only, no keys

Sensitive variables stay sensitive

```
variable "admin_password" {  
  sensitive = true  
}
```

Terraform should touch secrets, not hold them

Secrets are handled by letting Azure Key Vault store them and CI/CD use managed identity, so Terraform never exposes or hard-codes sensitive values.

9. You have multiple teams working on the same Terraform codebase targeting Azure. How do you prevent conflicts and ensure safe deployments?

Preventing conflicts & unsafe deployments

Reality check:

Terraform doesn't fail teams **teams fail Terraform.**

safety model:

Ownership by state, not repo

- Team A → networking state
- Team B → app state
- Team C → security state

No two teams write to same state.

Branch + pipeline enforcement

- Feature branches → plan only
- Main branch → apply
- Environment approvals for prod

Lock apply permissions

- Developers: plan
- Platform team: apply
- Prod requires approval

Policy as code

- Azure Policy
- Sentinel / OPA (if mature org)

Shared repo is fine. Shared state is chaos.

Conflicts are avoided by separating Terraform state by ownership, enforcing plan-before-apply through pipelines, and limiting who can apply changes to Azure environments.

10. A Terraform module needs a breaking change, but dozens of Azure environments depend on it. How do you roll this out safely?

Dozens of Azure environments depend on it

This is where **Terraform maturity shows**.

What I never do:

- Change module in place
- Force apply everywhere
- “Hope for the best”

Safe rollout strategy:

Version every module

source = "git::https://repo/modules/aks?ref=v1.3.0"

Release new major version

- v2.0.0 → breaking change
- v1.x stays untouched

Upgrade in waves

1. Dev
2. Staging

3. Low-risk prod
4. Critical prod

Migration flags (gold move)

```
variable "enable_new_behavior" {  
  default = false  
}
```

Gradual switch, zero downtime.

Modules evolve like APIs never like scripts

Breaking module changes are introduced through versioning and gradual upgrades, allowing existing Azure environments to remain stable while new behaviour is adopted safely.

11.How do you manage Terraform when Azure resources have different lifecycles (e.g., Azure SQL vs short-lived compute)?

(Azure SQL = long-lived, compute = short-lived)

The core principle:

Not all resources should move at the same speed.

Practical design approach:

Separate by lifecycle, not by service

- Long-lived:
 - Azure SQL
 - Key Vault
 - Networking
- Short-lived:
 - VMs
 - VM Scale Sets
 - App Services

Each lifecycle group:

- Has its **own module**

- Has its **own state**
- Is deployed independently

Protect critical resources

```
lifecycle {  
  prevent_destroy = true  
}
```

Used for:

- Databases
- Key Vaults
- Core networking

Faster iteration for compute

- Short-lived resources allow:
 - Frequent redeploys
 - Scale changes
 - Recreation without risk

Explanation sentence:

This approach treats Terraform like a lifecycle manager, keeping long-running Azure resources such as databases stable and protected, while allowing compute resources to change, scale, or be replaced without impacting critical systems.

12. Your Terraform run against Azure takes 20+ minutes. What are the common causes, and how do you optimize it?

Common causes & real optimization steps

Why this happens in Azure:

Too many resources in one state

- One massive state = slow refresh
- Every plan refreshes everything

Fix: Split state by domain or lifecycle.

Azure API throttling

- Parallel calls exceed limits
- Seen heavily with networking & RBAC

Fix:

```
terraform {  
  experiments = [module_variable_optional_attrs]  
}
```

and:

```
terraform apply -parallelism=5
```

Unnecessary refresh

Default behavior refreshes all resources.

Fix:

```
terraform plan -refresh=false  
(Only for trusted pipelines)
```

Provider version drift

New providers can slow API interactions.

Fix: Pin provider versions.

Slow Terraform is usually a design problem, not a performance problem.

A Terraform run usually becomes slow in Azure when too many resources share one state, API throttling kicks in, or unnecessary refresh happens, so optimization focuses on state design and controlled execution rather than faster machines.

13.How do you enforce policies like “no public Azure Storage Accounts” or mandatory resource tagging using Terraform?

Terraform alone is not enough enforcement is layered.

Layer 1: Terraform validation

```
variable "tags" {  
  validation {  
    condition    = contains(keys(var.tags), "owner")  
    error_message = "Owner tag is mandatory."  
  }  
}
```

Layer 2: Azure Policy (real enforcement)

- Deny public Storage Accounts
- Require tagging
- Block non-compliant resources

Terraform then **fails fast** when policy blocks deployment.

Layer 3: CI/CD guardrails

- terraform plan must pass
- Policy check before apply
- No manual exceptions in prod

Terraform suggests. Azure Policy enforces.

This solution ensures Terraform follows rules during deployment, but relies on Azure Policy to actually block non-compliant resources, so mistakes are prevented even if someone tries to bypass Terraform.

14. When would you use count vs for_each in Terraform, and what real issues have you faced by choosing the wrong one?

When to use each & real issues from wrong choice

Use count when:

- Resources are identical
- Order does not matter
- No external identity needed

count = 3

Use for_each when:

- Resources have unique names
- Lifecycle must be stable
- Partial removal is expected

`for_each = var.subnets`

Real problems caused by wrong choice:

Mistake	Impact
Using <code>count</code> for named resources	Resource recreation
Removing item from middle	Index shift destroys infra
Refactoring <code>count</code> → <code>for_each</code>	Forced recreation

If identity matters, `count` is dangerous.

The choice between `count` and `for_each` matters because `count` can accidentally destroy Azure resources when order changes, while `for_each` preserves identity and stability in real production environments.

15. You need to import existing Azure infrastructure into Terraform. What's your strategy to avoid downtime and unexpected changes?

Zero downtime, zero surprises strategy

Step 1: Observe before touching

- Document current Azure config
- No code changes yet

Step 2: Write Terraform to match reality

- Same names
- Same SKUs
- Same settings

No “improvements” during import.

Step 3: Import carefully

```
terraform import azurerm_resource.example <resource_id>
```

One resource at a time.

Step 4: Validate safely

```
terraform plan
```

Expected result:

No changes

If changes appear → stop and fix code.

Step 5: Lock critical resources

```
prevent_destroy = true
```

Import is reconciliation, not refactoring.

Importing existing Azure infrastructure into Terraform is done by first matching the code to the current environment exactly, then importing resources carefully so Terraform learns the setup without causing downtime or forced changes.

16. How do you design Terraform workflows to support blue-green or canary-style infrastructure changes on Azure?

Terraform is not the traffic switch Azure is.

How it works in practice:

Duplicate infrastructure, not modify in place

- Blue = current
- Green = new

Both exist simultaneously.

Terraform manages infra versions

- Separate modules or flags
- Separate state if needed

Azure controls traffic

- Azure Front Door
- Application Gateway
- Traffic Manager

Terraform:

- Creates infra
- Configures routing
- Does **not** cut traffic blindly

Canary approach

- Route 5% traffic
- Monitor
- Increase gradually

Terraform builds roads. Azure decides who drives on them.

Terraform is used to create parallel Azure environments safely, while Azure traffic services handle the switch, ensuring new versions are tested in production without risking user impact.